

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С. П. КОРОЛЕВА»

«САМАРСКИЙ УНИВЕРСИТЕТ»

Институт информатики и кибернетики
Кафедра геоинформатики и информационной безопасности

Отчет по лабораторной работе №1

Выполнил:
Железнов А.М.
гр. 6312-100503D
Проверил: Минаев Е.Ю.

Самара 2026

Задание на лабораторную работу 1:

Написать программу на языке C/C++ для перемножения двух квадратных матриц.

Исходные данные: файл(ы) содержащие значения исходных матриц.

Выходные данные: файл со значениями результирующей матрицы, время выполнения, объем задачи.

Обязательна автоматизированная верификация результатов вычислений с помощью сторонних библиотек или стороннего ПО (например на Matlab/Python).

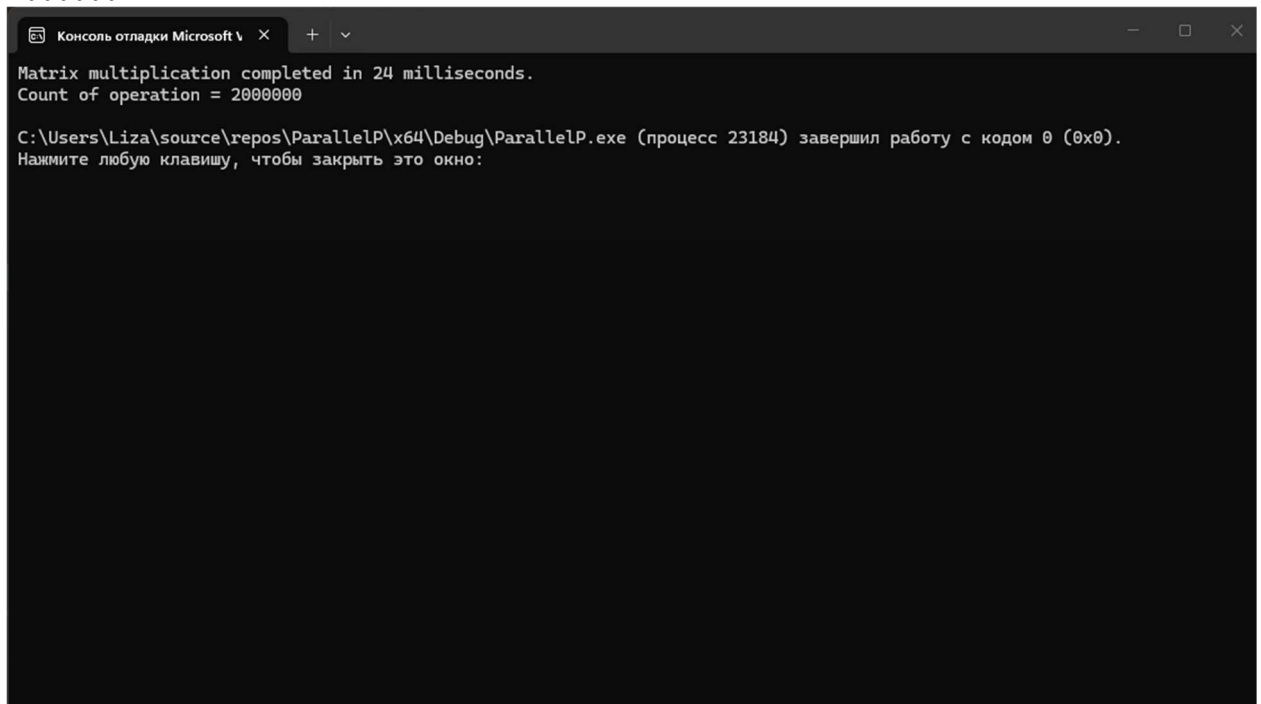
Ход работы:

В ходе лабораторной работы мною были сгенерированы 2 квадратные матрицы размером 100x100 элементов при помощи модуля generate_matrix.py. Были сгенерированы матрицы matrix_a и matrix_b

После генерации матриц, при помощи модуля main.cpp было выполнено умножение матриц matrix_a и matrix_b. В результате перемножения матриц я получил новую матрицу result_matrix.

Время выполнения функции mul_sq_matrix(...) составило «Matrix multiplication completed in 24 milliseconds.»

Количество операций для умножения матриц составило «Count of operation = 2000000»

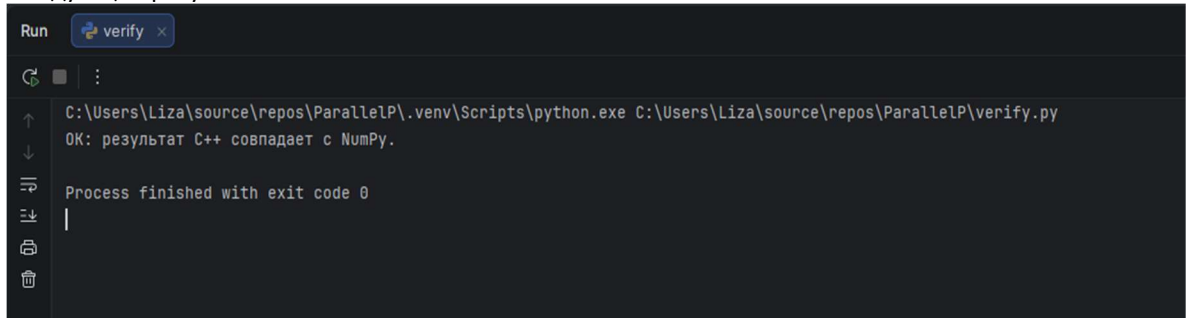


```
Консоль отладки Microsoft Visual Studio
Matrix multiplication completed in 24 milliseconds.
Count of operation = 2000000

C:\Users\Liza\source\repos\ParallelP\x64\Debug\ParallelP.exe (процесс 23184) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 1 - Результаты выполнения модуля main.cpp

Для верификации результатов я использовал модуль verify.py. По итогам работы этого модуля я получил следующие результаты:



```
Run verify x
C:\Users\Liza\source\repos\ParallelP\.venv\Scripts\python.exe C:\Users\Liza\source\repos\ParallelP\verify.py
OK: результат C++ совпадает с NumPy.
Process finished with exit code 0
```

Рисунок 2 - результат работы модуля verify.py

Итоги: в ходе лабораторной работы я изучил механизмы перемножения матриц на языке программирования C++, а также верифицировал результат работы моей программы

Исходные коды компонентов, а также матрицы можно посмотреть на моем github <https://github.com/Artem362a/ParallelProgrammingLabs/tree/lab1>, а также в этом документе (разместить матрицы в этом документе не удалось, они слишком большие)

Модули:

Generate matrix.py

```
import numpy as np

def random_matrix_100x100(low=0, high=10):
    # матрица 100x100 из целых в [low, high)
    return np.random.randint(low, high, size=(100, 100))
def save_matrix_to_file(matrix: np.ndarray, filename: str) -> None:
    """
    Сохраняет матрицу в текстовый файл:
    каждая строка матрицы – отдельная строка файла,
    элементы разделены пробелами.
    """
    np.savetxt(filename, matrix, fmt="%d", delimiter=" ")

if __name__ == "__main__":
    matrix_a = random_matrix_100x100()
    matrix_b = random_matrix_100x100()
    save_matrix_to_file(matrix_a, filename="matrix_a")
    save_matrix_to_file(matrix_b, filename="matrix_b")
```

main.cpp

```
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <chrono>
using namespace std;

using Matrix = vector<vector<int>>>;

Matrix read_sq_matrix_from_file(const string& filename) {
    ifstream in(filename);
    if (!in) {
        cerr << "Error opening file: " << filename << endl;
        return {};
    }
}
```

```

Matrix matrix;
string line;

while (getline(in, line)) {
    if (line.empty()) continue;

    istringstream iss(line);
    vector<int> row;
    int x;

    while (iss >> x) {
        row.push_back(x);
    }

    if (!row.empty())
        matrix.push_back(row);
}
return matrix;
}

void write_sq_matrix_to_file(const Matrix& matrix, const string& filename) {
    ofstream out(filename);
    if (!out) {
        throw runtime_error("Error opening file: " + filename);
    }

    int n = static_cast<int>(matrix.size());
    if (n == 0) return;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            out << matrix[i][j];
            if (j + 1 < n) out << ' ';
        }
        out << '\n';
    }
}

Matrix mul_sq_matrix(const Matrix& matrix_a, const Matrix& matrix_b){
    int n = static_cast<int>(matrix_a.size());
    Matrix result_matrix(n, vector<int>(n, 0));

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            int sum = 0;
            for (int k = 0; k < n; ++k) {
                sum += matrix_a[i][k] * matrix_b[k][j];
            }
            result_matrix[i][j] = sum;
        }
    }

    return result_matrix;
}

int main(){
    try{
        Matrix matrix_a = read_sq_matrix_from_file("matrix_a");
        Matrix matrix_b = read_sq_matrix_from_file("matrix_b");

        int n = static_cast<int>(matrix_a.size());
    }
}

```

```

        long long operations_counts = 2LL * n * n * n;

        auto start = chrono::high_resolution_clock::now();

        Matrix result_mul_matrix = mul_sq_matrix(matrix_a, matrix_b);

        auto end = chrono::high_resolution_clock::now();

        write_sq_matrix_to_file(result_mul_matrix, "result_matrix");
        auto duration = chrono::duration_cast<chrono::milliseconds>(end -
start).count();

        cout << "Matrix multiplication completed in " << duration << "
milliseconds." << endl;
        cout<<"Count of operation = "<<operations_counts<<endl;

        return 0;
    }
    catch (const exception& e) {
        cerr << "Error: " << e.what() << endl;
        return 1;
    }
}

```

verify.py

```

import numpy as np

def read_matrix_from_file(filename: str) -> np.ndarray:
    return np.loadtxt(filename, dtype=int)

def verify_cpp_result(a_file: str, b_file: str, c_file: str) -> None:
    A = read_matrix_from_file(a_file)
    B = read_matrix_from_file(b_file)
    C_cpp = read_matrix_from_file(c_file)

    C_py = A @ B

    if C_py.shape != C_cpp.shape:
        print("Размеры не совпадают!")
        print("C++:", C_cpp.shape, "Python:", C_py.shape)
        return

    if np.array_equal(C_py, C_cpp):
        print("ОК: результат C++ совпадает с NumPy.")
    else:
        print("ОШИБКА: результаты отличаются.")
        print("Разность (Python - C++):")
        print(C_py - C_cpp)

if __name__ == "__main__":
    verify_cpp_result("matrix_a", "matrix_b", "result_matrix")

```